Chapter 4

Semantic Analysis

Assume that the program has been verified to be syntactically correct and converted into some kind of intermediate representation (a parse tree). The next phase will be **semantic analysis** of the generated parse tree. Semantic analysis also includes **error reporting in case any semantic error is found out.**

- **Semantic analysis** is a pass by a compiler that adds semantic information to the **parse tree** and performs certain checks based on this information. It logically follows the parsing phase, in which the parse tree is generated, and logically precedes the code generation phase, in which (intermediate/target) code is generated. (In a compiler implementation, it may be possible to fold different phases into one pass.) Typical examples of semantic information that is added and checked is typing information (**type checking**) and the binding of variables and function names to their definitions (**object binding**). *Sometimes also some early code optimization is done in this phase*.

Following things are done in Semantic Analysis:

- **Disambiguate Overloaded operators**: If an operator is overloaded, one would like to specify the meaning of that particular operator because from one will go into code generation phase next.
- **Type checking** : The process of verifying and enforcing the constraints of types is called type checking. This may occur either at **compile-time** (a static check) or run-time (a dynamic check). **Static type checking** is a primary task of the semantic analysis carried out by a compiler. If type rules are enforced strongly (that is, generally allowing only those automatic type conversions which do not lose information), the process is called strongly typed, if not, weakly typed.
- **Uniqueness checking** : Whether a variable name is unique or not, in the its scope.
- **Name Checks** : Check whether any variable has a name which is not allowed.

❖ Beyond syntax analysis

- Parser cannot catch all the program errors
- There is a level of correctness that is deeper than syntax analysis
- Some language features cannot be modeled using context free grammar formalism

A parser has its own limitations in catching program errors related to semantics, something that is deeper than syntax analysis. Typical features of semantic analysis cannot be modeled using context free grammar formalism. If one tries to incorporate those features in the definition of a language then that language does not remain context free anymore.

Example 1

- ❖ **string** x; int y; y = x + 3; The use of x is a type error. (a string can't add with integer)
- ❖ **int** a, b; a = b + c; Here, **c** is not declared
- ❖ **int** x; **char** x; An identifier x refer to different data types and makes declaration conflicts
- ❖ A variable declared within one function cannot be used within the scope of the definition of the other function unless declared there separately.

These are a couple of examples, which tell us that typically what a compiler has to do beyond syntax analysis. This is just an example. Probably you can think of many more examples in which syntax analysis will not handle.

Compiler needs to know?

- ☞ Whether a variable has been declared?
- ☞ What is the type of the variable?
- ☞ Whether a variable is a scalar, an array, or a function?
- ☞ What declaration of the variable does each reference use?
- ☞ If an expression is type consistent?
- ☞ If an array use like A[i,j,k] is consistent with the declaration? Does it have three dimensions?
- ☞ How many arguments does a function take?
- ☞ Are all invocations of a function consistent with the declaration?
- ☞ If an operator/function is overloaded, which function is being invoked?
- ☞ Inheritance relationship

If the compiler has the answers to all these and other questions only, then will it be able to successfully do a semantic analysis by using the generated parse tree.

How to answer these questions?

In order to answer the previous questions the compiler will have to keep information about the **type of variables**, **number of parameters in a particular function, types of inheritance used** etc. It will have to do some sort of computation in order to gain this information. Most compilers keep a structure called **symbol table to store** this information.

How to . ?

In **syntax analysis** we used context free grammar. Here we put lot of attributes around it. So it consists of **context sensitive grammars** along with **extended attribute grammars**. Formalism may be so difficult that writing specifications itself may become tougher than writing compiler itself. So we use attributes we can do analysis along with parse tree itself instead of using context sensitive grammars.

Attribute grammar is nothing but it is a CFG and attributes put around all the terminal and non-terminal symbols are used. Despite the difficulty in the implementation of the attribute grammar formalism it has certain big advantages which makes it desirable.

- **An attribute grammar** is the formal expression of the syntax-derived semantic checks associated with a grammar.
- It represents the rules of a language not explicitly imparted by the syntax.
- In a practical way, it defines the information that will need to be in the abstract syntax tree in order to successfully perform semantic analysis.
- This information is stored as attributes of the nodes of the abstract syntax tree.
- The values of those attributes are calculated by semantic rule.

There are two ways for writing attributes:

1) **Syntax Directed Definition** : It is a high level specification in which implementation details are hidden. Details like at what point of time is it evaluated and in what manner are hidden from the programmer.

2) **Translation scheme**: Sometimes we want to control the way the attributes are evaluated, the order and place where they are evaluated. This is of a slightly lower level and allow some implementation details to be shown.

Example of Attribute grammar:

E → E + T { E.value = E.value + T.value }

The right part of the CFG contains the semantic rules that specify how the grammar should be interpreted. Here, the values of non-terminals E and T are added together and the result is copied to the non-terminal E.

Semantic attributes may be assigned to their values from their domain at the time of parsing and evaluated at the time of assignment or conditions**. Based on the way the** attributes get their values, they can be broadly divided into two categories: **synthesized attributes and inherited attributes.**

- **Synthesized attributes:** These attributes get values from the attribute values of their child nodes. To illustrate, assume the following production:

  S → ABC

If S is taking values from its child nodes (A, B, C), then it is said to be a synthesized attribute, as the values of ABC are synthesized to S.

As in our previous example (E → E + T), the parent node E gets its value from its child node. ***Synthesized attributes*** **never take** values from their parent nodes or any sibling nodes.

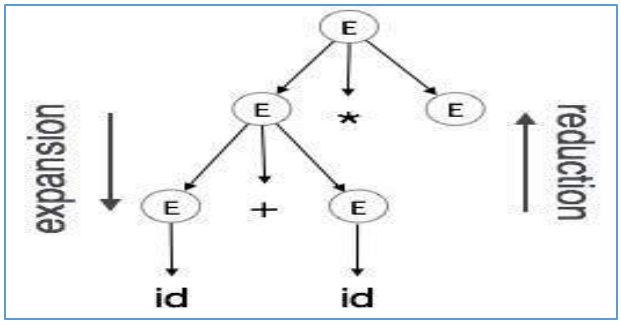
- **Inherited attributes:** In contrast to synthesized attributes, inherited attributes can take values from parent and/or siblings. As in the following production,

  S → ABC

A can get values from S, B and C. B can take values from S, A, and C. Likewise, C can take values from S, A, and B.

Expansion: When a non-terminal is expanded to terminals as per a grammatical rule.

Reduction: When a terminal is reduced to its corresponding non-terminal according to grammar rules.



Semantic analysis uses Syntax Directed Translations (SDT) to perform the above tasks.

Syntax Directed Translation. has augmented rules to the grammar that facilitate semantic analysis. SDT involves passing information bottom-up and/or top-down to the parse tree in form of attributes attached to the nodes.

- There are Two types of SDT: These are **S-attributed SDT** and **L-attributed STD**

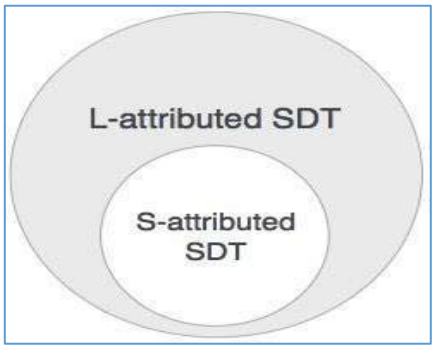
A. S-attributed SDT

If an SDT uses only synthesized attributes, it is called as S-attributed SDT. These attributes are evaluated using S-attributed SDTs that have their semantic actions written **after the production (right hand side)**. S-attributed SDTs are evaluated in bottom-up parsing, as the values of the **parent nodes** depend upon the values of the **child nodes.**

B. L-attributed SDT

This form of SDT uses both **synthesized** and **inherited** attributes with restriction of not taking values from **right siblings.** Semantic action rules can be placed anywhere in right side**.** In L-attributed SDTs, a non-terminal can get values from its **parent**, **child**, and sibling nodes. As in the following production $S \rightarrow ABC$

S can take values from **A**, **B**, and **C** (synthesized). **A** can take values from **S** only. **B** can take values from **S** and **A**. **C** can get values from **S**, **A**, and **B**. No non-terminal can get values from the sibling to its right. Attributes in L-attributed SDTs are evaluated by depth-first and left-to-right parsing manner.



From the diagram, we can say that, All S-attributed STD are L-attributed STD.

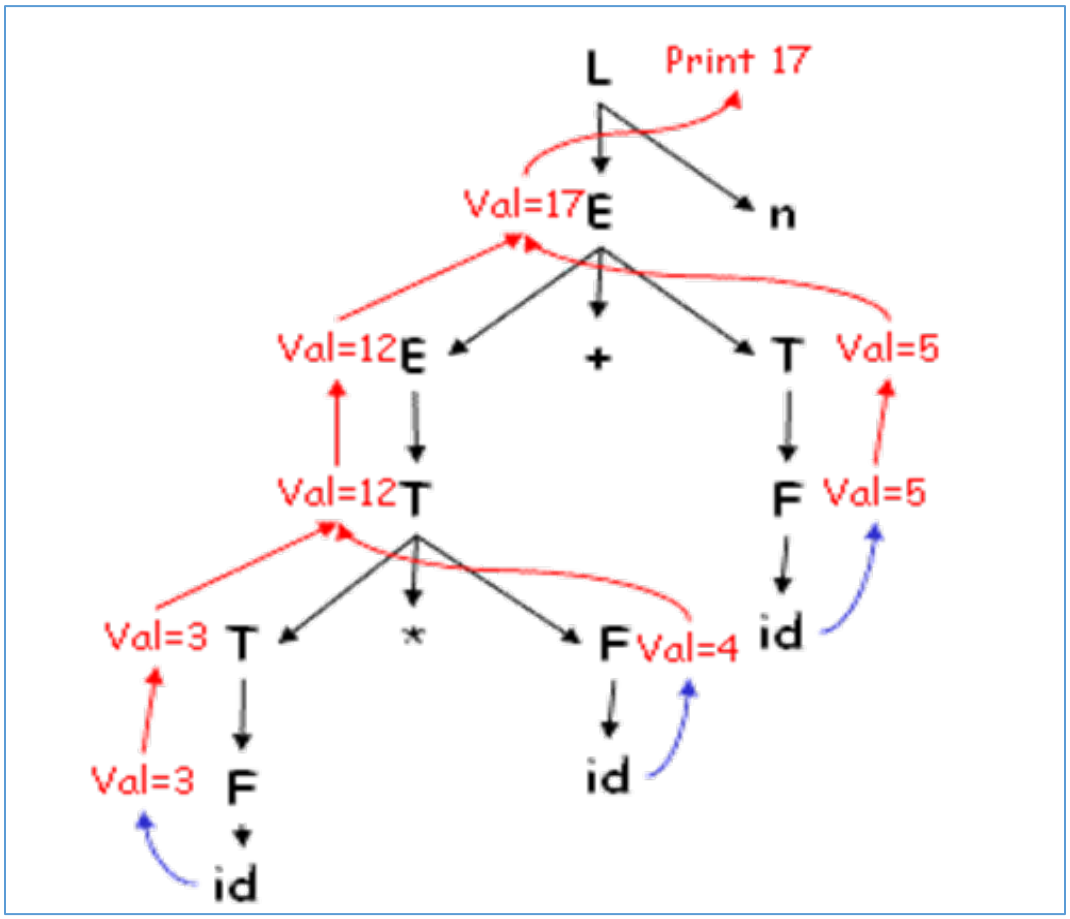
Example for S-attributed SDT by synthesized attribute

Syntax Directed Definitions for a desk calculator program

Productions	Semantic Rules
L ⟶ E n	Print (E.val) (**n** indicates new line)
E ⟶ E + T	E.val = E.val + T.val
E ⟶ T	E.val = T.val
T ⟶ T * F	T.val = T.val * F.val
T ⟶ F	T.val = F.val
F ⟶ (E)	F.val = E.val
F ⟶ digit	F.val = digit.lexval (digit is can have only lexical meaning)

- terminals are assumed to have only synthesized attribute values of which are supplied by lexical analyzer
- Start symbol has no parents, hence no inherited attributes.

Example Parse tree for 3 * 4 + 5 n *here n is newline*



Using the previous attribute grammar calculations have been worked out here for 3 * 4 + 5 n. Bottom up parsing has been done.

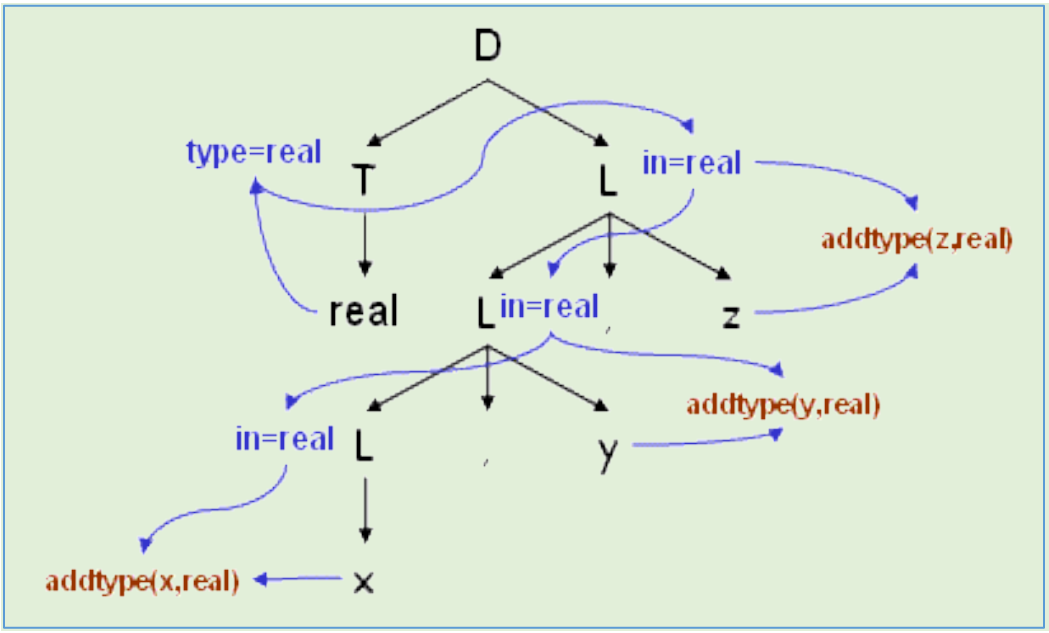
Example for L-attributed SDT by inherited attribute

Syntax Directed Definitions for a declaration

D ⟶ T L	L.in = T.type
T ⟶ real	T.type = real
T ⟶ int	T.type = int
L ⟶ L_1 , id	L_1 .in = L.in; addtype(id.entry, L.in)
L ⟶ id	addtype (id.entry,L.in)

- ✓ Inherited attributes help to find the context (type, scope etc.) of a token e.g., the type of a token or scope when the same variable name is used multiple times in a program in different functions.
- ✓ Here addtype(id, L.in) functions adds a symbol table entry for the id a and attaches to its parent the type of L.in .

Parse tree for **real x, y, z**



Dependence of attributes in an inherited attribute system. The value of in (an inherited attribute) at the three L nodes gives the type of the three identifiers x , y and z . These are determined by computing the value of the attribute T.type at the left child of the root and then valuating L.in top down at the three L nodes in the right subtree of the root. At each L node the procedure addtype is called which inserts the type of the identifier to its entry in the symbol table. The figure also shows the dependence graph which is introduced later.

❖ Dependence Graph

It is directed graph indicating interdependencies among the **synthesized** and **inherited** attributes of various nodes in a parse tree.

- ☞ If an attribute b depends on an attribute c then the semantic rule for b must be evaluated after the semantic rule for c
- ☞ The dependencies among the nodes can be depicted by a directed graph called dependency graph

Algorithm to construct dependency graph

for each node **n** in the parse tree do

    for each attribute **a** of the grammar symbol do

        construct a node in the dependency graph

            for **a**

for each node n in the parse tree do

    for each semantic rule b = f (c_1 , c_2 , …, c_k) do

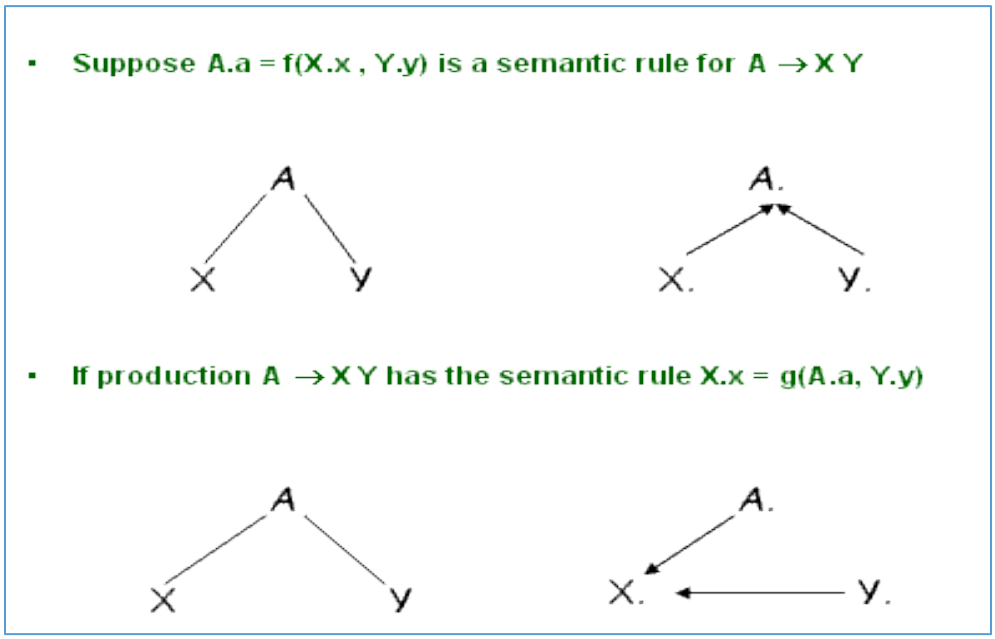
    { associated with production at n }

        for i = 1 to k do

            construct an edge from c_i to b

❖ An algorithm to construct the dependency graph is : After making one node for every attribute of all the nodes of the parse tree, make one edge from each of the other attributes on which it depends.

Example

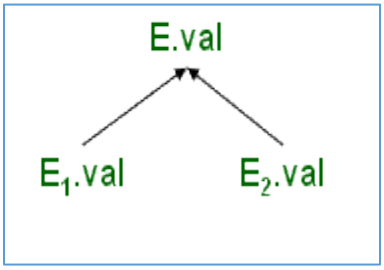


The semantic rule A.a = f(X.x , Y.y) for the production A -> XY defines the **synthesized attribute** a of A to be dependent on the attribute x of X and the attribute y of Y . Thus the dependency graph will contain an edge from X.x to A.a and Y.y to A.a accounting for the two dependencies. Similarly for the semantic rule X.x = g(A.a , Y.y) for the same production there will be an edge by **inherited attribute** from A.a to X.x and an edg e from Y.y to X.x.

Example

Consider the following production: $E \rightarrow E1 + E2$ E.val = E1 .val + E 2 .val

then a dependency graph will be



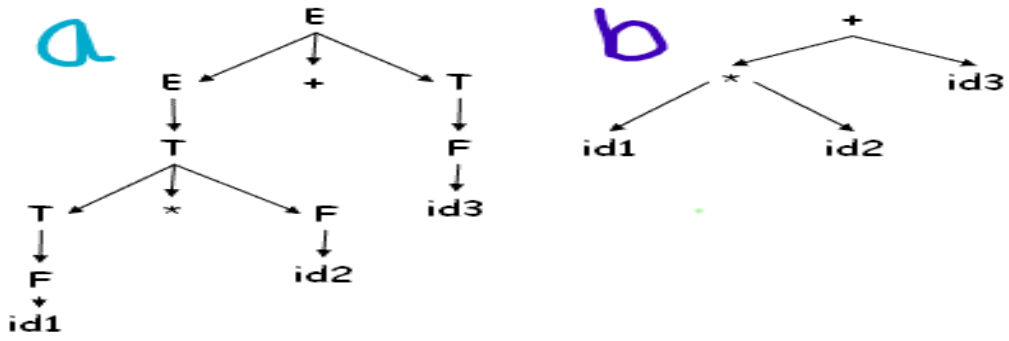
- The synthesized attribute E.val depends on E1.val and E2.val hence the two edges one each from E 1 .val & E 2 .val

Abstract Syntax Tree

An Abstract Syntax Tree (syntax tree) is a tree in which each leaf node represents an **operand**, while each inside node represents an **operator**. The syntax is "abstract" in the sense that it does not represent every detail appearing in the real syntax, but rather just the structural or content-related details. It is condensed form of parse trees. The syntax tree is usually used when representing a program in a tree structure.

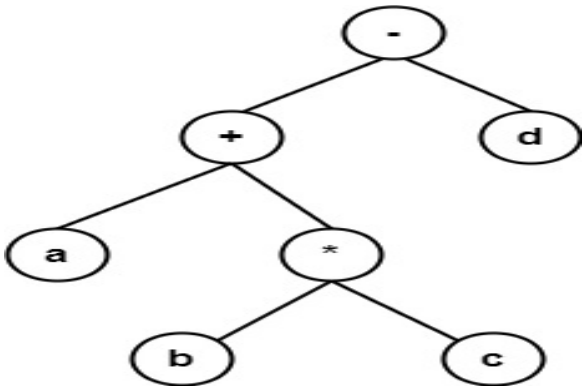
It is usually the result of the syntax analysis phase of a compiler. It often serves as an intermediate representation of the program through several stages that the compiler requires, and has a strong impact on the final output of the compiler.

☞ Consider the following tree (**a**) is parse tree (b) is Abstract syntax tree



Chain of single production are collapsed into one node with the operators moving up to become the node.

Example 2 Draw Syntax Tree for the string **a + b ∗ c − d**.



Rules for constructing a syntax tree

Each node in a syntax tree can be executed as data with multiple fields. In the node for an operator, one field recognizes the operator and the remaining field includes a pointer to the nodes for the operands. The operator is known as the label of the node. The following functions are used to create the nodes of the syntax tree for the expressions with binary operators. Each function returns a pointer to the recently generated node.

- **mknode (op, left, right)** − It generates an operator node with label op and two field including pointers to left and right.
- **mkunode(op, entry)-** It generates urinary operator for labels
- **mkleaf (id, entry)** − It generates an identifier node with label id and the field including the entry, a pointer to the symbol table entry for the identifier.
- **mkleaf (num, val)** − It generates a number node with label num and a field including val, the value of the number.

For example : Let us have the following SDT

E→E+T { E.ptr = **mknode**('+',E.ptr , T.ptr);}

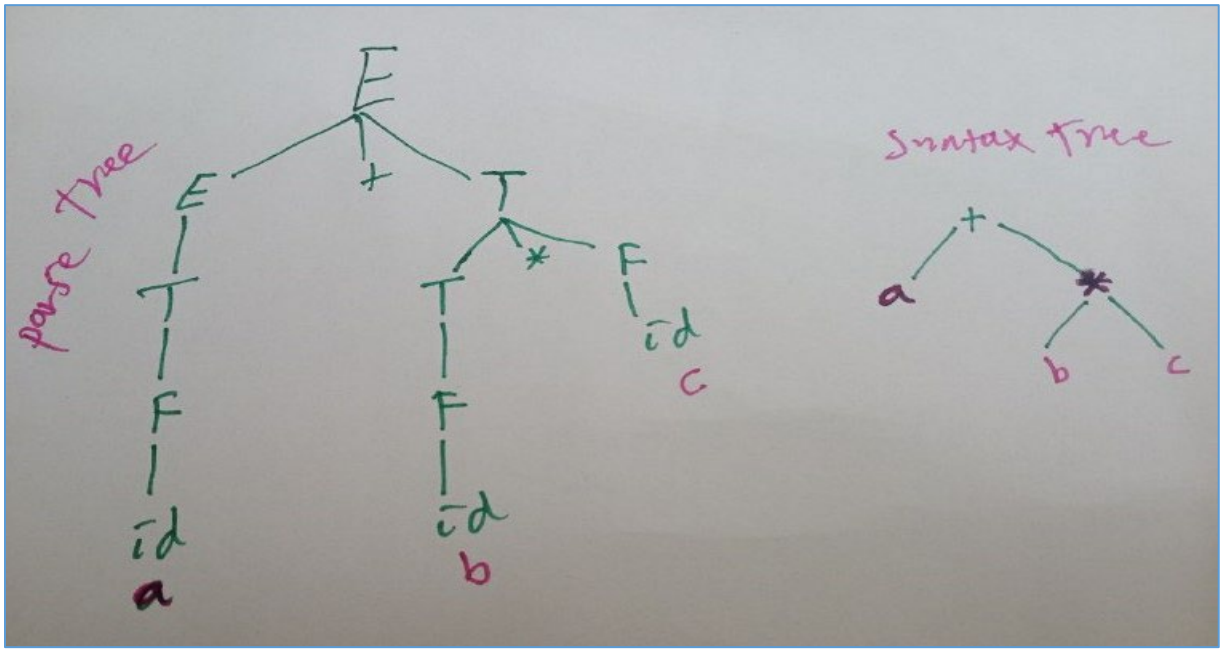
E→T {E.ptr=T.ptr ;}

T→T*F {T.ptr = **mknode** ('*' , T.ptr, F.ptr) ;}

T→F {T.ptr = F.ptr ;}

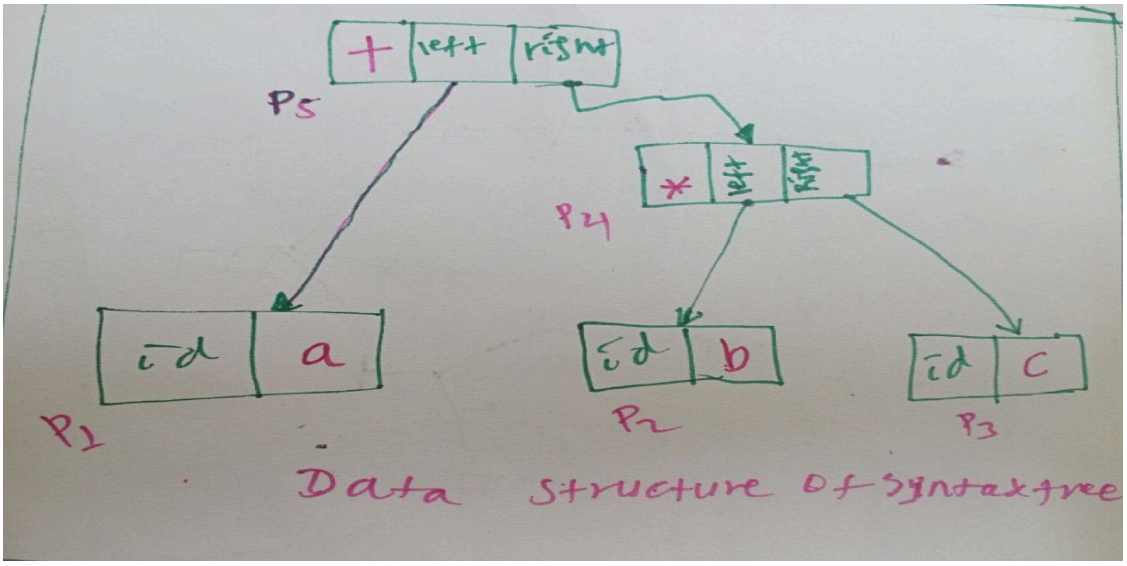
F→id {F.ptr = **mkleaf** (id, id.entry ;)

Using the above SDT (Both production rule and Semantic Actions) let us parse the tree having **a+b*c**

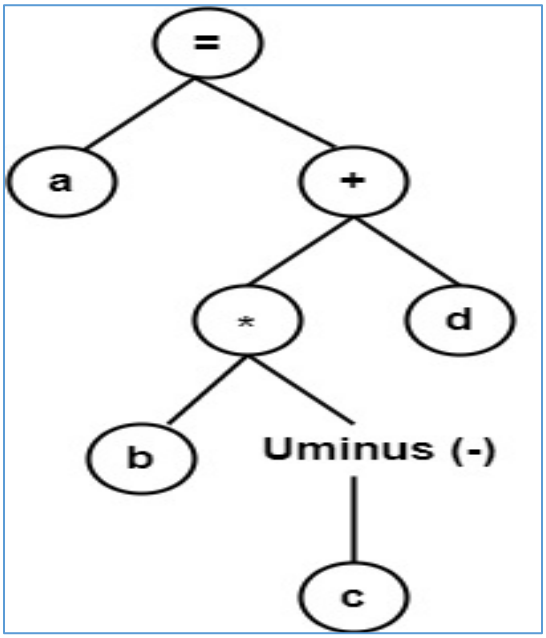


To compute the semantic action for the first production rule E→E+T { E.ptr = **mknode**('+',E.ptr , T.ptr);} First we have to compute for T, and again to solve T we have to compute F first. F can be solved directly because it directs to an **id.** This indicates it is done in bottom up manner. p_1, p_2, … . . p_5 are pointers to the symbol table entries for identifier 'a' 'b', and 'c' respectively.

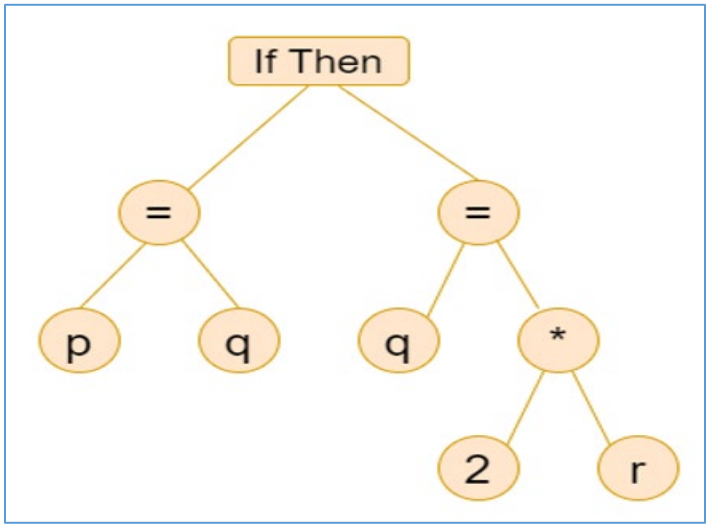
p1 : mkleaf(id, id.entry) : id , a
p2 : mkleaf (id, id.entry**):** id, b
p3: mkleaf(id, id.entry) : id, c
p4 : mknode ('*' , T.ptr, F.ptr): *, p2, p3
p5: mknode ('+',E.ptr, T,ptr**)** : +, p1,p4



Example 2 syntax tree for the expression. **a = b ∗ −c + d**



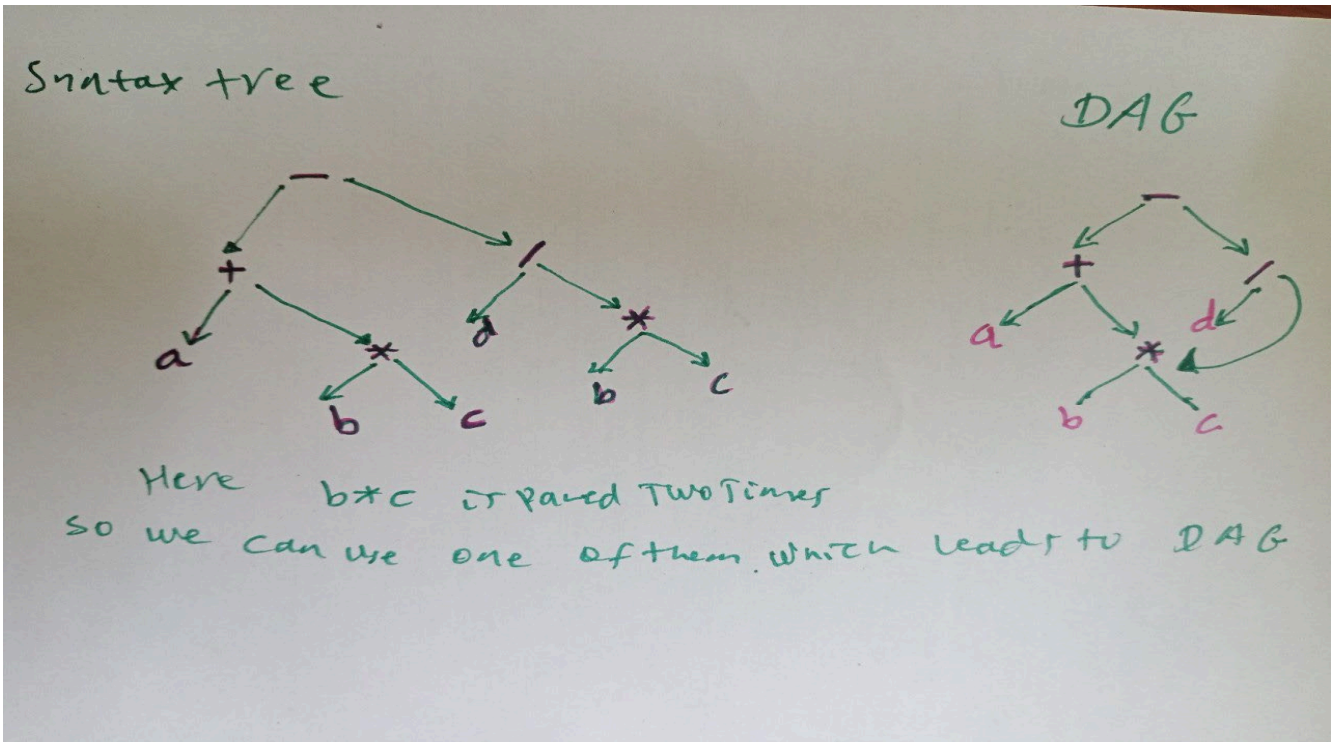
Example 3 syntax tree for a statement. **If p = q then q = 2 * r**



Directed Acyclic Graph (DAG)

A directed acyclic graph (DAG) is on **Abstract syntax tree**, All the function calls are made as in the order and Whenever the required node is already present, a pointer to it is returned so that a pointer to the old node itself is obtained. A new node is made if it did not exist before.

Example Let us Construct DAG for the following expression **(a+b*c) –(d/(b*c))**



Type system

Type checking is an important aspect of semantic analysis. A type is a set of values. Certain operations are valid for values of each type. For example, consider the type integer in C++. The operation **mod**(%) can only be applied to values of type integer, and to no other types. A language's type specifies which operations are valid for values of each type. **A type checker** verifies that the type of a construct matches, which is expected by its context, and ensures that correct operations are applied to the values of each type. Languages can be classified into three main categories depending upon the type system they employ. **These are :**

- **Untyped** : In these languages, there are no explicit types. Allows any operation to be performed on any data . Tcl, BCPL, *Assembly languages* are category of these languages.
- **Statically typed** : In these type of languages, all the type checking is done at the compile time only. This means that before source code is compiled, the type associated with each and every single variable must be known. Because of this, these languages are also called Strongly typed languages. Example of languages in this category are Algol ,C++, java, Kotlin and Scala etc.
- **Dynamically typed** : In dynamically typed languages, the type checking is done at the runtime. This means that variables are checked against types only when the program is executing. Usually, functional programming languages like Lisp, PHP, JavaScript etc. have dynamic type checking.

Type Systems: is a collection of rules that assign types to program constructs (more constraints added to checking the validity of the programs, violation of such constraints indicate errors). The implementation of a type system is a **type checker**. Type systems provide a concise formalization of the semantic checking rules. These rules are defined on the structure of expressions and are language specific. Different compilers or processors of the same language may use different type systems.

Type Expressions

A **type expression** is either a basic type or is formed by applying an operator called a type constructor to a type expression. The sets of basic types and constructors depend on the language to be checked.

The following are some of **type expressions:**

- **A basic type** is a type expression. Typical basic types for a language include boolean, char, integer, float, and void (the absence of a value.
- ***Sub-range* types**: A sub-range type defines a range of values within the range of another type. For example, type A = 1..10; B = 100..1000; U = 'A'..'Z';

- ***Enumerated* types:** An enumerated type is defined by listing all of the possible values for the type. For example: type Colour = (Red, Yellow, Green); Both the sub-range and enumerated types can be treated as basic types.
- **A constructor type expressions and it includes:**

  - **Arrays** : If T is a type expression, then array(I, T) is a type expression denoting the type of an array with elements of type T and index set I. I is often a range of integers. Ex. int a[25] ;
  - **Products** : If T_1 and T_2 are type expressions, then their Cartesian product T_1 x T_2 is a type expression. x associates to the left and that it has higher precedence. Products are introduced for completeness; they can be used to represent a list or tuple of types (e.g., for function parameters).
  - **Records** : A record is a data structure with named fields. A type expression can be formed by applying the record type constructor to the field names and their types.
  - **Pointers** : If T is a type expression, then pointer (T) is a type expression denoting the type "pointer to an object of type T". For example: **int a; int *p=&a;**
  - **Functions**: Mathematically, a function maps depends on one set (domain) to another set(range). **Function F : D -> R.** A type expression can be formed by using the type constructor **->** for **function** types. We write s -> t for "function from type s to type t".

Type Conversion

If a data type is automatically converted into another data type at compile time is known as type conversion. The conversion is performed by the compiler if both data types are compatible with each other. Consider expression like **x + i** where **x** is of type real and **i** is of type integer. Since, the internal representations of **integers** and **reals** are different in a computer The compiler has to convert both the operands to the same type. Language definition specifies what conversions are necessary and performed. Usually conversion is to the type of **the left hand side.** Thus, Type checker is used to insert conversion operations: x + i to **x** real+ inttoreal **(i)**.

Consider the following Production rules with semantic action

```
E → num          E.type = int
E → num.num      E.type = real
E → id           E.type = lookup( id.entry
E → E 1 op E 2   E.type = if E 1 .type == int && E 2 .type == int
                                                  then int
                 elseif E 1 .type == int && E 2 .type == real
                                                  then real
                 elseif E 1 .type == real && E 2 .type == int
                                                  then real
                 elseif E 1 .type == real && E 2 .type== real
                                                  then real
```

The language has to perform implicit type conversions where ever it is possible. As a rule compiler performs a type conversion if it doesn't lead to any loss of information. For example, **int** can be converted into **real**, single precision to a double precision etc. under this rule. Compilers also support explicit type conversions. This conversion translates into many cases in the type assignment of a Left Hand side of a expression. For example, x = y + z ; Then the type of x depends on the type of y, z as described above.